

Lab Submission: Linux System Commands and C Integration

Author: Phạm Nguyễn Anh Huy

System: Arch Linux (Hyprland)

1. Command Manipulation & Syntax

1.1. `man`

Purpose: Displays the reference manual for system commands.

Syntax: `man [command]`

- **Example 1:** `man ls` – Displays the manual for the list directory contents command.
- **Example 2:** `man grep` – Displays the manual for the pattern-matching utility.

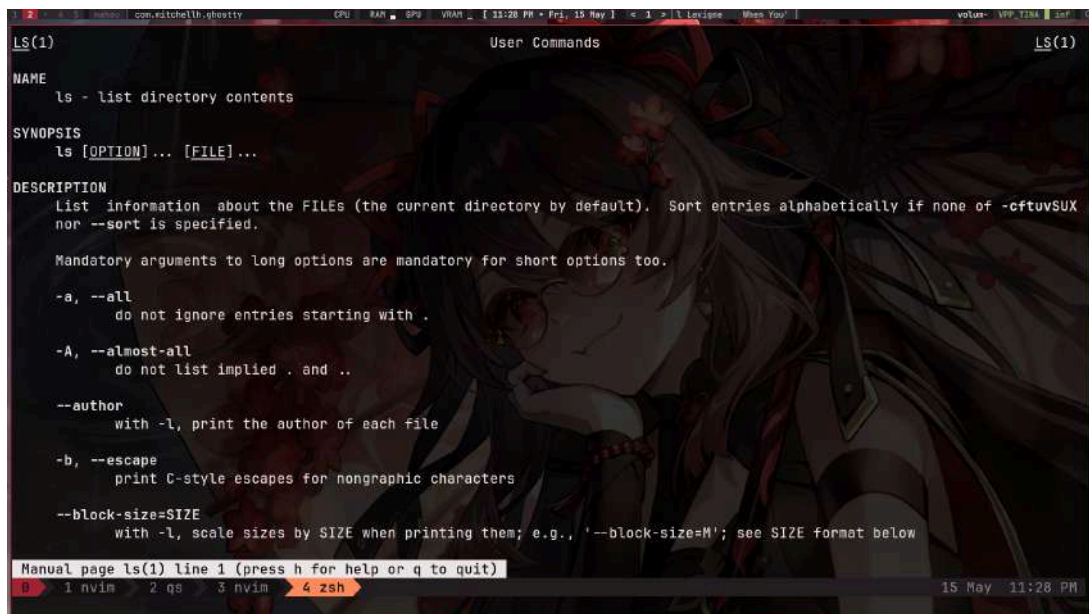
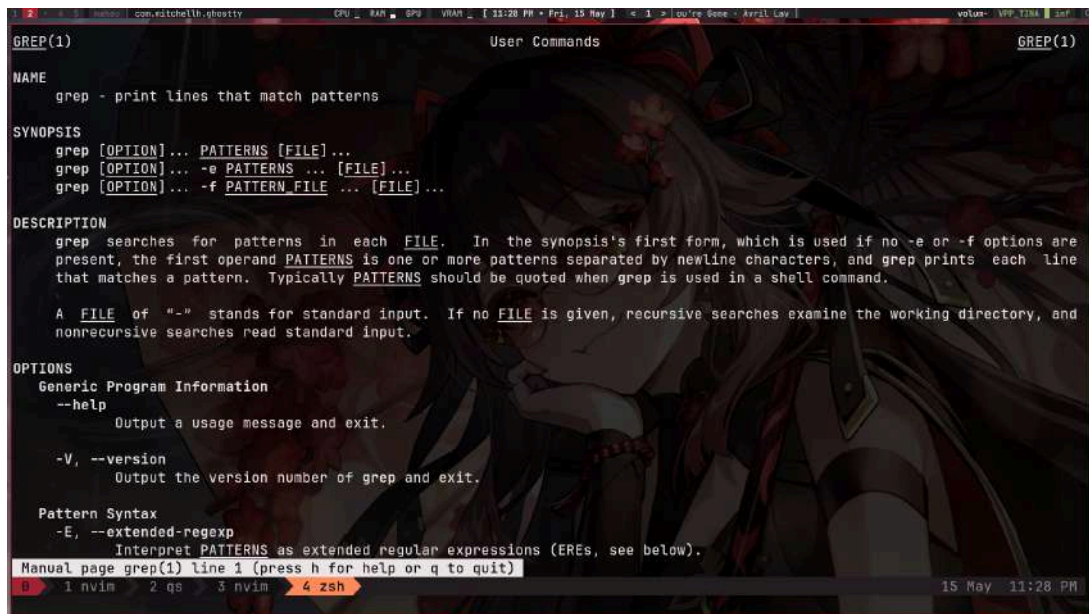


Figure 1: Example 1 – `man ls`



```
GREP(1) User Commands GREP(1)
NAME
  grep - print lines that match patterns
SYNOPSIS
  grep [OPTION]... PATTERNS [FILE]...
  grep [OPTION]... -e PATTERNS ... [FILE]...
  grep [OPTION]... -f PATTERN_FILE ... [FILE]...
DESCRIPTION
  grep searches for patterns in each FILE. In the synopsis's first form, which is used if no -e or -f options are
  present, the first operand PATTERNS is one or more patterns separated by newline characters, and grep prints each
  line that matches a pattern. Typically PATTERNS should be quoted when grep is used in a shell command.
  A FILE of "-" stands for standard input. If no FILE is given, recursive searches examine the working directory, and
  nonrecursive searches read standard input.
OPTIONS
  Generic Program Information
  --help
    Output a usage message and exit.
  -V, --version
    Output the version number of grep and exit.
  Pattern Syntax
  -E, --extended-regexp
    Interpret PATTERNS as extended regular expressions (EREs, see below).
Manual page grep(1) line 1 (press h for help or q to quit)
0 1 nvim 2 qs 3 nvim 4 zsh
15 May 11:28 PM
```

Figure 2: Example 2 – `man grep`

1.2. `bg` & `fg`

Purpose: `bg` moves a suspended process to the background; `fg` brings a background process back to the foreground.

Syntax: `bg %[job_id]` / `fg %[job_id]`

- **Example 1:** Running `sleep 1000`, pressing `Ctrl+Z`, then typing `bg %1`.
- **Example 2:** Typing `fg %1` to bring that sleep process back to the active terminal.

A terminal window with a dark background and a large anime-style illustration of a girl with long hair and a red flower in her hair. The terminal shows a user named 'tsubenn' at 11:27 PM. They run 'sleep 1000' and press ^Z, which suspends the process. They then run 'jobs' and see '[1] + suspended sleep 1000'. Pressing ^C resumes it as '[1] + continued sleep 1000'. Finally, they run 'fg %1' and the process runs as '[1] + running sleep 1000'. At the bottom, a taskbar shows four windows: 1 nvim, 2 qs, 3 nvim, and 4 sleep (highlighted in orange). The system clock at the bottom right shows '15 May 11:27 PM'.

```
tsubenn (11:27PM): ~ $ sleep 1000
^Z
zsh: suspended sleep 1000
tsubenn (11:27PM): ~ $ jobs
[1] + suspended sleep 1000
tsubenn (11:27PM): ~ $ bg %1
[1] + continued sleep 1000
tsubenn (11:27PM): ~ $ fg %1
[1] + running sleep 1000
```

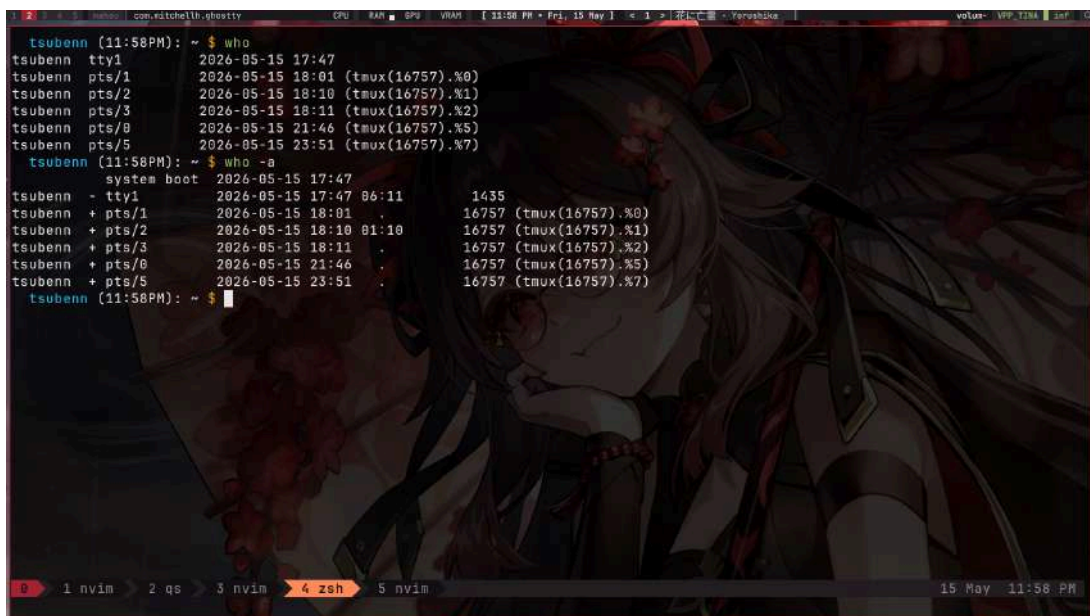
Figure 3: Examples – `bg` and `fg`

1.3. `who`

Purpose: Displays a list of users currently logged into the system.

Syntax: `who [options]`

- **Example 1:** `who` – Basic list of logged-in users.
- **Example 2:** `who -a` – Displays all available information for current users.

A terminal window with the same background as Figure 3. The user 'tsubenn' at 11:58 PM runs 'who', which shows a list of logged-in users: 'tsubenn' on 'tty1' at 17:47, and several 'pts' sessions on 'tmux(16757)' at various times. Then, they run 'who -a', which shows more detailed information, including the system boot time (2026-05-15 17:47) and the user's login history (e.g., 'system boot', 'tty1', 'pts/1', etc.). At the bottom, the taskbar now shows five windows: 1 nvim, 2 qs, 3 nvim, 4 zsh (highlighted in orange), and 5 nvim. The system clock shows '15 May 11:58 PM'.

```
tsubenn (11:58PM): ~ $ who
tsubenn tty1      2026-05-15 17:47
tsubenn pts/1      2026-05-15 18:01 (tmux(16757).%0)
tsubenn pts/2      2026-05-15 18:10 (tmux(16757).%1)
tsubenn pts/3      2026-05-15 18:11 (tmux(16757).%2)
tsubenn pts/0      2026-05-15 21:46 (tmux(16757).%5)
tsubenn pts/5      2026-05-15 23:51 (tmux(16757).%7)
tsubenn (11:58PM): ~ $ who -a
system boot 2026-05-15 17:47
tsubenn - tty1 2026-05-15 17:47 06:11 1435
tsubenn + pts/1 2026-05-15 18:01 . 16757 (tmux(16757).%0)
tsubenn + pts/2 2026-05-15 18:10 01:10 16757 (tmux(16757).%1)
tsubenn + pts/3 2026-05-15 18:11 . 16757 (tmux(16757).%2)
tsubenn + pts/0 2026-05-15 21:46 . 16757 (tmux(16757).%5)
tsubenn + pts/5 2026-05-15 23:51 . 16757 (tmux(16757).%7)
tsubenn (11:58PM): ~ $
```

Figure 4: Examples – `who` and `who -a`

1.4. `chown` , `chgrp` , `chmod`

Purpose: Used to manage file ownership (`chown`), group ownership (`chgrp`), and access permissions (`chmod`).

Syntax: `chmod [mode] [file]` , `chown [owner] [file]` ,
`chgrp [group] [file]`

- `chmod` – Grants owner full access and others read/execute.
- `chown` – Changes file owner to the user `tsubenn` .
- `chgrp` – Changes file group to `wheel` .



```
tsubenn (11:44PM): ~/OS6202 $ touch my_script.sh
tsubenn (11:44PM): ~/OS6202 $ ls
my_script.sh
tsubenn (11:44PM): ~/OS6202 $ ls -l
total 0
-rw-r--r-- 1 tsubenn tsubenn 0 May 15 23:44 my_script.sh
tsubenn (11:45PM): ~/OS6202 $ chmod -x my_script.sh
tsubenn (11:48PM): ~/OS6202 $ ls -l
total 0
-rw-r--r-- 1 tsubenn tsubenn 0 May 15 23:44 my_script.sh
tsubenn (11:48PM): ~/OS6202 $ touch private.txt
tsubenn (11:49PM): ~/OS6202 $ ls -l
total 0
-rw-r--r-- 1 tsubenn tsubenn 0 May 15 23:44 my_script.sh
-rw-r--r-- 1 tsubenn tsubenn 0 May 15 23:49 private.txt
tsubenn (11:49PM): ~/OS6202 $ chmod 600 private.txt
tsubenn (11:49PM): ~/OS6202 $ ls -l
total 0
-rw-r--r-- 1 tsubenn tsubenn 0 May 15 23:44 my_script.sh
-rw----- 1 tsubenn tsubenn 0 May 15 23:49 private.txt
tsubenn (11:49PM): ~/OS6202 $ sudo chown tsubenn private.txt
[sudo] password for tsubenn:
tsubenn (11:49PM): ~/OS6202 $ sudo chown tsubenn:wheel private.txt
tsubenn (11:50PM): ~/OS6202 $ sudo chgrp wheel thoca.txt
chgrp: cannot access 'thoca.txt': No such file or directory
tsubenn (11:50PM): ~/OS6202 $ touch thoca.txt
tsubenn (11:50PM): ~/OS6202 $ sudo chgrp wheel thoca.txt
tsubenn (11:50PM): ~/OS6202 $ sudo chgrp -R wheel ../Study
tsubenn (11:50PM): ~/OS6202 $
```

Figure 5: `chmod` , `chown` , `chgrp` examples

1.5. `date`

Purpose: Displays or sets the system date and time.

Syntax: `date [+format]`

- **Example 1:** `date` – Shows current date and time.
- **Example 2:** `date +%D` – Shows the date in MM/DD/YY format.



Figure 6: Examples – `date`

1.6. `su`, `top`, `kill`, `jobs`

Purpose: System management tools for switching users (`su`), monitoring tasks (`top`), ending processes (`kill`), and listing shell jobs (`jobs`).

Syntax: `su [user]`, `top`, `kill [PID]`, `jobs`

- `su` – Change user.
- `top` – View real-time CPU/RAM usage.
- `jobs` – List background jobs with their process IDs.
- `kill` – Terminate a process by PID.

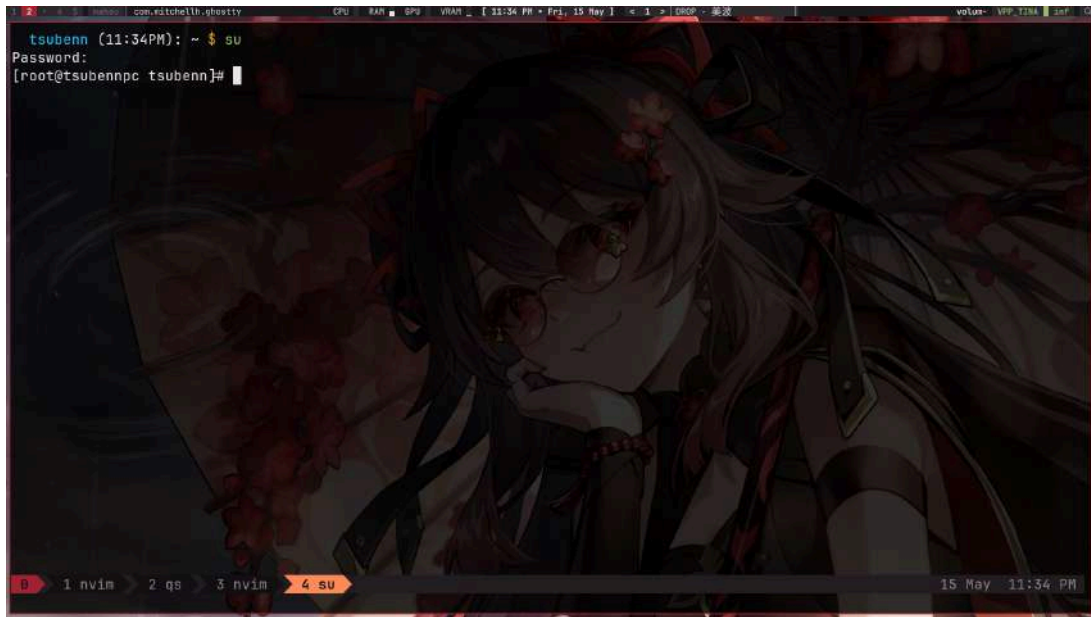


Figure 7: `su` example



Figure 8: `jobs` example

```
tsubenn (11:38PM): ~ $ ps
PID TTY          TIME CMD
294451 pts/0      00:00:03 zsh
419938 pts/0      00:00:00 dolphin
420176 pts/0      00:00:00 ps
tsubenn (11:39PM): ~ $ kill -l
HUP INT QUIT ILL TRAP IOT BUS FPE KILL USR1 SEGV USR2 PIPE ALRM TERM STKFLT CHLD CONT STOP TSTP TIOC TIOU URG XCPU XFSZ VTALRM
PROF WINCH POLL PWR SYS
tsubenn (11:39PM): ~ $ kill -s KILL 419938
tsubenn (11:39PM): ~ $
[1] + killed  dolphin
tsubenn (11:39PM): ~ $
```

Figure 9: `kill` example

```
top - 23:34:41 up 5:47, 1 user, load average: 0.18, 0.35, 0.52
Tasks: 373 total, 1 running, 371 sleep, 0 d-sleep, 0 stopped, 1 zombie
%Cpu(s):  1.6 us,  0.8 sy,  0.0 ni, 81.6 id, 15.7 wa,  0.4 hi,  0.0 si,  0.0 st
MiB Mem : 15874.6 total, 2620.3 free, 6612.3 used, 6444.9 buff/cache
MiB Swap: 40305.0 total, 38104.0 free, 2201.0 used. 8462.4 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM    TIME+  COMMAND
 1456 tsubenn   20    0 1091648 156796 130680 S   4.8   1.0   5:01.75 Hypnland
 1924 tsubenn   20    0 1399.2g 398552 97804 S   4.8   2.6 16:30.95 vesktop
 16691 tsubenn  20    0 2313660 301100 252264 S   4.8   2.0  4:24.37 ghostty
365896 tsubenn  20    0 1939720 661592 309712 S   4.8   4.3  1:15.77 qs
415537 tsubenn  20    0  11316   7532   5380 R   4.8   0.0   0:00.01 top
   1 root      20    0  26032  10948   8028 S   0.0   0.1   0:01.56 systemd
   2 root      20    0      0      0      0 S   0.0   0.0   0:00.00 kthreadd
   3 root      20    0      0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
   4 root      0 -20    0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
   5 root      0 -20    0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
   6 root      0 -20    0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
   7 root      0 -20    0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
   8 root      0 -20    0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
  11 root      0 -20    0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-kblockd
  14 root      0 -20    0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
  15 root      20    0      0      0      0 S   0.0   0.0   0:00.23 ksoftirqd/0
  16 root      -2    0      0      0      0 S   0.0   0.0   0:04.58 rcu_preempt
  17 root      -2    0      0      0      0 S   0.0   0.0   0:00.00 rcub/0
  18 root      20    0      0      0      0 S   0.0   0.0   0:00.00 rcu_exp_par_gp_kthread_worker/0
  19 root      20    0      0      0      0 S   0.0   0.0   0:00.00 rcu_exp_gp_kthread_worker
  20 root      rt    0      0      0      0 S   0.0   0.0   0:00.59 migration/0
  21 root      20    0      0      0      0 S   0.0   0.0   0:00.00 kprobe-optimizer
  22 root     -51    0      0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
```

Figure 10: `top` example

2. Pattern Searching with Grep

Command executed:

```
grep "nhu nui" thoca.txt
```



```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int ret;

    // Example 1: Calling the 'man' page for 'ls'
    ret = system("man ls");

    // Example 2: Calling the 'date' command (commented out)
    // system("date");

    return ret;
}

```

```

c > linux.c > 5: main
1 #include<stdlib.h>
2
3 int main() {
4     int ret;
5
6     ret = system(command: "man ls");
7     return ret;
8
linux.c [+] 8,1 All
1 nvim 2 qs 3 nvim 4 nvim 5 nvim 16 May 12:07 AM

```

Figure 12: Source code

```
tsubenn (12:08AM): ~/dotfiles/programs/c/linux $ gcc -c linux.c -o linux.o
tsubenn (12:08AM): ~/dotfiles/programs/c/linux $ ls -l
total 8
-rw-r--r-- 1 tsubenn tsubenn 93 May 16 00:07 linux.c
-rw-r--r-- 1 tsubenn tsubenn 1496 May 16 00:08 linux.o
tsubenn (12:08AM): ~/dotfiles/programs/c/linux $
```

Figure 13: Compiled object file

```
tsubenn (12:12AM): ~/dotfiles/programs/c/linux $ ./linux
```

Figure 14: Before running the binary

```
LS(1) User Commands LS(1)
NAME
ls - list directory contents
SYNOPSIS
ls [OPTION]... [FILE]...
DESCRIPTION
List information about the FILES (the current directory by default). Sort entries alphabetically if none of -eftuvSUX
nor --sort is specified.

Mandatory arguments to long options are mandatory for short options too.

-a, --all
do not ignore entries starting with .
-A, --almost-all
do not list implied . and ..

--author
with -l, print the author of each file
-b, --escape
print C-style escapes for nongraphic characters

--block-size=SIZE
with -l, scale sizes by SIZE when printing them; e.g., '--block-size=M'; see SIZE format below

-B, --ignore-backups
do not list implied entries ending with ~
Manual page ls(1) line 1 (press h for help or q to quit)
0 1 nvim 2 qs 3 nvim 4 zsh 5 nvim 16 May 12:12 AM
```

Figure 15: After running the binary